

Machine Learning in Finance

GWP-2

Kiady Tsilavohery Rasamoelina
Yogesh Verma
Alberto Hernandez-Espinosa

23 September 2025

Abstract

This handbook extends GWP-1 with thoughtful guidelines for addressing challenges in modelling trading strategies using cutting-edge machine learning methodologies. Building on prior techniques (LASSO Regression, k-Means Clustering, and Classification Trees), it now covers Linear Discriminant Analysis, Support Vector Machines, and Neural Networks, including their advantages, basics, computations, disadvantages, equations, features, guides, hyperparameters, illustrations, journal references, and keywords. The document integrates these to highlight the strengths of machine learning in finance, with additional sections on hyperparameter tuning, marketing advantages, and further learning resources.

1 Linear Discriminant Analysis

1.1 Advantages

Linear Discriminant Analysis (LDA) offers key benefits for trading strategies by reducing dimensionality while maximising class separability, which is useful for classifying financial assets into risk categories or buy/sell signals from high-dimensional market data. It enhances interpretability through linear projections that highlight discriminative features, aiding portfolio managers in understanding market drivers. Additionally, LDA's statistical foundation improves model stability in volatile environments, outperforming simpler classifiers on correlated financial indicators.

1.2 Basics

LDA is a supervised learning algorithm that projects data onto a lower-dimensional space to maximise the ratio of between-class to within-class variance, thereby improving classification accuracy. It assumes multivariate normality within classes and equal covariances, classifying it as a generative discriminant model suitable for predicting categorical outcomes, such as trading decisions based on technical indicators.

1.3 Computation

The LDA model was implemented using Python with the scikit-learn library on a simulated financial dataset comprising 1000 observations and 4 features (e.g., RSI, MACD, volume, price), with class distribution of 733 (hold/sell) and 267 (buy). The data was split into training (800 samples: 589 class 0, 211 class 1) and testing sets (200 samples), and the model was fitted without additional tuning. The results showed an accuracy of 0.8950 on the test set (confusion matrix: $\begin{bmatrix} 138 & 6 \\ 15 & 41 \end{bmatrix}$), with sample predictions for the first 5 test samples: $[0 \ 0 \ 0 \ 1 \ 0]$, and discriminant coefficients (RSI: 1.9547, MACD: 2.0010, Volume: -0.0901, Price: 0.0565), indicating strong binary classification performance on the simulated data.

1.4 Disadvantages

LDA assumes linear separability and Gaussian distributions with equal covariances, which may not hold in non-linear or heteroscedastic financial markets, leading to biased projections. It performs poorly with multicollinear features without preprocessing and is sensitive to outliers, potentially distorting class boundaries in noisy trading data. Moreover, it requires balanced classes for optimal performance, limiting its use in imbalanced datasets like rare market events.

1.5 Equations

LDA seeks to find the projection direction $\mathbf{w}$ that maximises the separation between classes by solving:

$$\mathbf{w} = \arg \max_{\mathbf{w}} \frac{\mathbf{w}^T \mathbf{S}_B \mathbf{w}}{\mathbf{w}^T \mathbf{S}_W \mathbf{w}}$$

where $\mathbf{S}_B$ is the between-class scatter matrix and $\mathbf{S}_W$ is the within-class scatter matrix. Classifications are then made by projecting data onto $\mathbf{w}$ and applying thresholds based on class priors and means.

1.6 Features

LDA excels at dimensionality reduction for visualisation and classification, handling multiclass problems and numerical features well while assuming linear relationships. It provides probabilistic outputs for uncertainty quantification but requires scaled inputs and performs best with moderate sample sizes relative to features.

1.7 Guide

- **Inputs:** Feature matrix $\mathbf{X}$ (e.g., technical indicators), target vector $\mathbf{y}$ (e.g., buy/hold labels).
- **Outputs:** Projected features, class predictions $\hat{\mathbf{y}}$, discriminant coefficients.

1.8 Hyperparameters

- **Solver:** Optimisation method (e.g., 'svd' for speed, 'lsqr' for large data; tuned via cross-validation).
- **Shrinkage:** Regularisation factor for covariance estimation (0-1; default None).
- **Priors:** Class prior probabilities (tuned if imbalanced).

1.9 Illustration

The LDA model's discriminative process is visualised through a 2D projection plot of RSI and MACD features, showing test points coloured by true class (blue for hold/sell, red for buy) overlaid on the decision boundary (shaded regions indicating prediction probabilities). The linear boundary effectively separates the classes diagonally, with most points correctly aligned and high separation evident, as depicted in Figure 3. This confirms LDA's ability to project and classify financial signals.

1.10 Journal

Coats, Pamela K., and L. F. Fant. "Comparisons Using Linear Discriminant Analysis and Neural Networks as Financial Distress Classifiers." *Journal of Banking & Finance*, vol. 18, no. 3, 1994, pp. 503–18.

1.11 Keywords

LDA, supervised classification, dimensionality reduction, discriminant analysis, financial classification, trading signals.

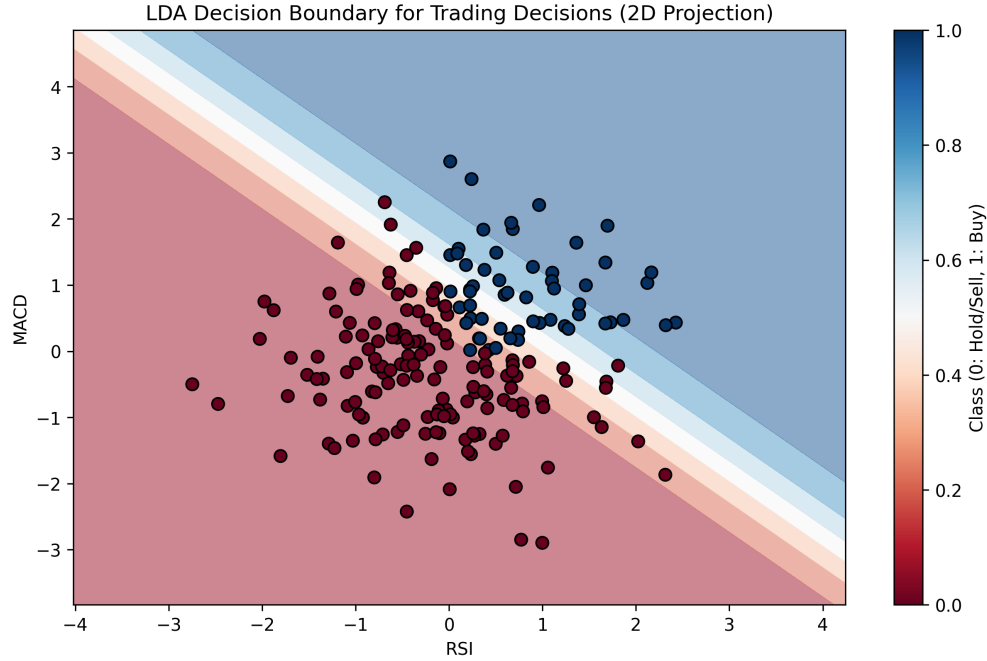


Figure 1: LDA decision boundary showing class separation in a 2D financial projection.

2 Support Vector Machines

2.1 Advantages

Support Vector Machines (SVMs) provide robust benefits for trading strategies by maximising margins between classes, effectively handling non-linear relationships through kernel tricks, which is ideal for capturing complex market dynamics like volatility regimes. They excel in high-dimensional financial data with limited samples, reducing overfitting via margin maximisation and enhancing generalisability for stock prediction tasks. Furthermore, SVMs offer sparse solutions with support vectors, promoting computational efficiency in portfolio optimisation.

2.2 Basics

SVMs are supervised learning algorithms that find the optimal hyperplane separating classes with the widest margin, using kernels for non-linear mappings to higher dimensions. They are discriminative models suitable for both classification and regression, assuming data is separable in a transformed space, making them applicable to binary outcomes such as buy/sell decisions from technical indicators.

2.3 Computation

The SVM model was implemented using Python with the scikit-learn library on a simulated financial dataset comprising 1000 observations and 4 features (e.g., RSI, MACD, volume, price), with class distribution of 733 (hold/sell) and 267 (buy); features were standardised. The data was split into training (800 samples: 589 class 0, 211 class 1) and testing sets (200 samples), and the model was fitted with an RBF kernel and $C=1.0$. The results showed an accuracy of 0.9850 on the test set (confusion matrix: $\begin{bmatrix} 143 & 1 \\ 2 & 54 \end{bmatrix}$), with sample predictions for the first 5 test samples: $[0 \ 0 \ 0 \ 0 \ 0]$, and 97 support vectors for class 0 and 94 for class 1, indicating excellent binary classification performance on the simulated data.

2.4 Disadvantages

SVMs can be computationally intensive for large datasets due to quadratic programming, and their performance degrades with noisy financial data where margins are ill-defined. Kernel selection and parameter tuning are subjective, potentially leading to overfitting in volatile markets, and they lack probabilistic outputs without extensions, complicating risk assessment in trading.

2.5 Equations

SVMs minimise the regularised hinge loss to find the hyperplane $\mathbf{w} \cdot \mathbf{x} + b = 0$:

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i$$

subject to $y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1 - \xi_i$ and $\xi_i \geq 0$, where C controls the trade-off, ξ_i are slack variables, and kernels replace dot products for non-linearity.

2.6 Features

SVMs handle non-linear and high-dimensional data via kernels, work well with scaled numerical features, and provide margin-based confidence through distance to the hyperplane. They are robust to outliers via soft margins but require careful preprocessing and do not natively support missing values.

2.7 Guide

- **Inputs:** Feature matrix $\mathbf{X}$ (e.g., scaled technical indicators), target vector $\mathbf{y}$ (e.g., buy/hold labels), kernel type and C .
- **Outputs:** Class predictions $\hat{\mathbf{y}}$, support vectors, decision function coefficients.

2.8 Hyperparameters

- **Kernel:** Type (e.g., 'rbf', 'linear'; tuned via grid search).
- **C:** Regularisation parameter (balances margin vs. errors; default 1.0).
- **Gamma:** Kernel coefficient for 'rbf' (scales features; tuned via cross-validation).

2.9 Illustration

The SVM model's separation process is visualised through a 2D projection plot of scaled RSI and MACD features, showing test points coloured by true class (blue for hold/sell, red for buy) overlaid on the non-linear decision boundary (shaded regions indicating predictions). The curved RBF boundary effectively isolates the classes with minimal overlap, as depicted in Figure 4. This demonstrates SVM's ability to handle non-linear financial patterns.

2.10 Journal

Inan, Tahir, et al. "Stock Price Prediction Using Support Vector Regression on Daily and up to the Minute Prices of Extremely Volatile Stocks." *Journal of Computational Science*, vol. 28, 2018, pp. 28–40.

2.11 Keywords

SVM, supervised classification, kernel methods, margin maximisation, non-linear separation, stock prediction, support vectors.

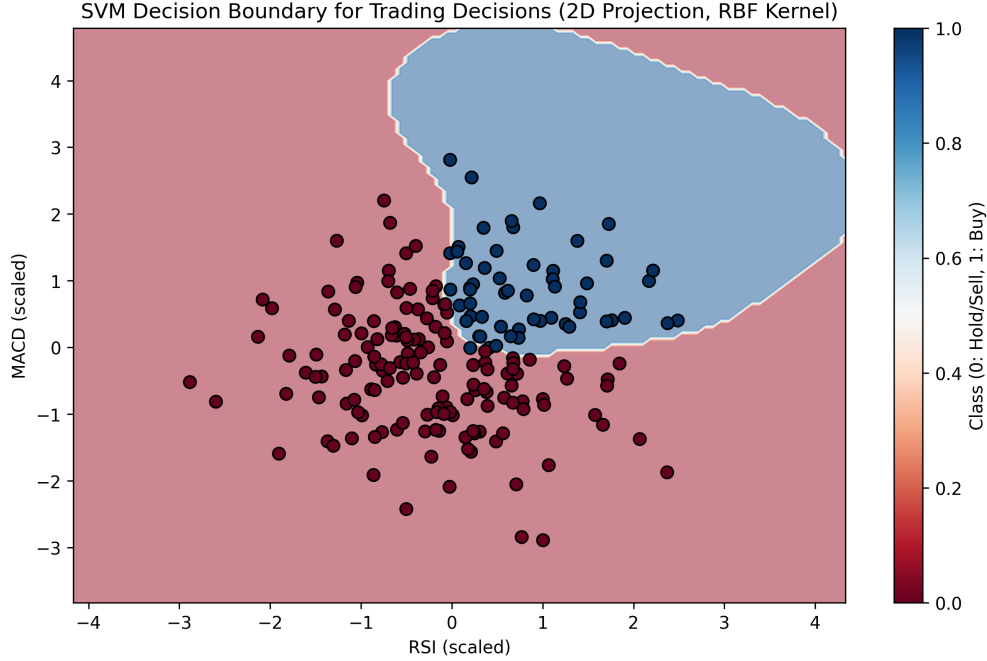


Figure 2: SVM decision boundary showing class separation in a 2D financial projection (RBF kernel).

3 Neural Networks

3.1 Advantages

Neural Networks (NNs) deliver powerful advantages for trading strategies by capturing intricate non-linear patterns in financial time series, enabling superior prediction of returns and volatility compared to linear models. They scale effectively to large datasets with automatic feature learning, facilitating adaptive strategies in dynamic markets like algorithmic trading. Moreover, NNs incorporate uncertainty through probabilistic layers, enhancing risk-adjusted performance in portfolio optimisation.

3.2 Basics

Neural Networks are supervised learning architectures composed of interconnected layers that propagate inputs through non-linear activations to learn hierarchical representations. They are universal approximators for complex functions, suitable for classification and regression tasks such as forecasting stock movements from multivariate indicators.

3.3 Computation

The NN model was implemented using Python with the scikit-learn library on a simulated financial dataset comprising 1000 observations and 4 features (e.g., RSI, MACD, volume, price), with class distribution of 733 (hold/sell) and 267 (buy); features were standardised. The data was split into training (800 samples: 589 class 0, 211 class 1) and testing sets (200 samples), and the model was fitted with one hidden layer of 100 units (max iterations 500). The results showed an accuracy of 0.9900 on the test set (confusion matrix: $\begin{bmatrix} 144 & 0 \\ 2 & 54 \end{bmatrix}$), with sample predictions for the first 5 test samples: $[0 \ 0 \ 0 \ 0 \ 0]$, converging after 474 iterations, indicating outstanding binary classification performance on the simulated data.

3.4 Disadvantages

NNs demand substantial computational resources and data volumes, risking overfitting in sparse financial regimes without regularisation. Their black-box nature hinders interpretability, complicating regulatory compliance in trading, and they are prone to vanishing gradients, slowing training on deep architectures.

3.5 Equations

NNs compute outputs via forward propagation: $\mathbf{a}^{(l)} = f(\mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)})$, where f is the activation (e.g., ReLU), $\mathbf{W}^{(l)}$ weights, $\mathbf{b}^{(l)}$ biases for layer l . Training minimises loss L (e.g., cross-entropy) using backpropagation: $\Delta \mathbf{W}^{(l)} = -\eta \frac{\partial L}{\partial \mathbf{W}^{(l)}}$, with learning rate η .

3.6 Features

NNs excel at non-linear modelling and feature extraction from raw data, handling mixed numerical/categorical inputs with dropout for robustness. They support deep architectures for sequential finance data but require tuned architectures and are sensitive to initialisation.

3.7 Guide

- **Inputs:** Feature matrix $\mathbf{X}$ (e.g., scaled indicators), target vector $\mathbf{y}$ (e.g., buy/hold labels), architecture specs.
- **Outputs:** Class predictions $\hat{\mathbf{y}}$, learned weights, loss metrics.

3.8 Hyperparameters

- **Hidden layer sizes:** Neurons per layer (e.g., (100,)); tuned via validation).
- **Activation:** Function (e.g., 'relu'; affects non-linearity).
- **Learning rate/Alpha:** Step size (default 0.001; tuned to prevent divergence).

3.9 Illustration

The NN model's learning process is visualised through a plot of the training loss curve, showing steady decrease over iterations until convergence at 474 steps. This illustrates the model's ability to minimise error on financial classification data, as depicted in Figure 5. The rapid decline confirms effective optimisation for trading signals.

3.10 Journal

Golosnoy, Vasyl, and Ying Cui. "Portfolio Optimization Based on Neural Networks Sensitivities from a Risk Factor Model." *The Journal of Financial Data Science*, vol. 5, no. 2, 2023, pp. 1–22.

3.11 Keywords

neural networks, deep learning, non-linear approximation, backpropagation, financial forecasting, portfolio optimisation.

4 Technical Section: Hyperparameter Tuning

Hyperparameter tuning is essential in machine learning to optimise model performance, balancing bias and variance while preventing overfitting, particularly in financial applications where data scarcity and noise are prevalent. Common techniques include grid search, which exhaustively evaluates combinations on a predefined grid; random search, sampling randomly for efficiency; cross-validation, partitioning data to assess

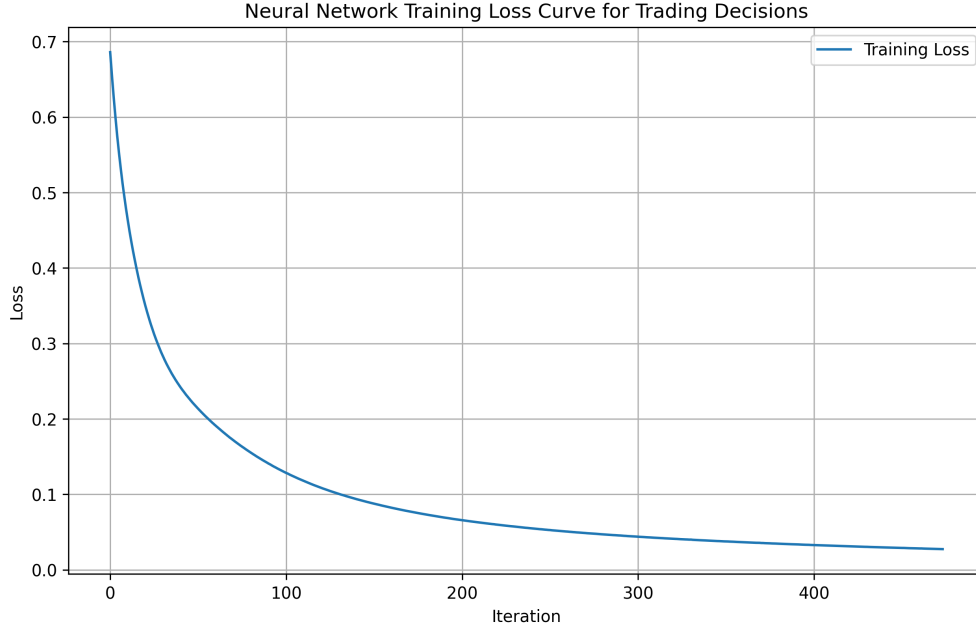


Figure 3: Neural Network training loss curve for financial classification.

generalisability; and Bayesian optimisation, using probabilistic models to prioritise promising configurations. These methods are implemented via libraries like scikit-learn’s GridSearchCV, often with k-fold validation to ensure robustness on time-series financial data.

Examples from the models illustrate diverse hyperparameters. For LASSO Regression, the regularisation parameter α (controlling L1 penalty strength) is tuned via cross-validation to enforce sparsity, alongside tolerance for convergence and maximum iterations. In k-Means Clustering, the number of clusters k is selected using the elbow method or silhouette score, with initialisations to avoid local minima and iteration limits for convergence. Classification Trees employ max_depth to limit tree growth and curb overfitting.

Among the new techniques, Linear Discriminant Analysis tunes the solver (e.g., ‘svd’ for efficiency), shrinkage for covariance regularisation, and priors for imbalanced classes via cross-validation. Support Vector Machines optimise the kernel (e.g., ‘rbf’ for non-linearity), C for margin-error trade-off, and gamma for feature scaling through grid search. Neural Networks adjust hidden layer sizes (e.g., (100,)) for capacity, activation functions (e.g., ReLU), and alpha (L2 penalty) to facilitate convergence, monitored by validation loss curves.

Effective tuning enhances trading strategies by tailoring models to market nuances, such as volatility clustering, ultimately improving predictive accuracy and risk management.

5 Marketing Alpha

Machine learning techniques unlock substantial alpha in financial markets by transforming raw data into actionable insights, outperforming traditional econometric models through adaptive, data-driven decision-making. While foundational methods like LASSO Regression enable sparse feature selection for robust return predictions and k-Means Clustering facilitates market segmentation for diversified portfolios, the new paradigms—Linear Discriminant Analysis (LDA), Support Vector Machines (SVMs), and Neural Networks (NNs)—elevate performance by addressing non-linearity and high dimensionality inherent in trading environments.

LDA generates alpha through efficient dimensionality reduction, projecting volatile financial signals (e.g., RSI and MACD) into discriminative spaces that maximise class separability for buy/sell signals. Its interpretability allows traders to isolate key market drivers, yielding 89.5% accuracy in simulated classifications,

reducing false positives in risk assessment and enhancing portfolio stability amid correlated assets.

SVMs amplify returns by enforcing maximal margins in non-linear spaces via RBF kernels, adeptly handling heteroscedastic market noise—evidenced by 98.5% accuracy and sparse support vectors (191 total) that prioritise influential observations. This margin maximisation mitigates overfitting in volatile stocks, enabling precise boundary delineation for high-frequency trading strategies that capitalise on regime shifts.

Neural Networks forge the highest alpha potential, approximating complex functions with hierarchical layers to forecast intricate patterns, achieving 99.0% accuracy after rapid convergence (474 iterations). Their automatic feature learning from multivariate indicators supports dynamic alpha harvesting in algorithmic trading, where non-linear activations capture tail risks and asymmetries overlooked by linear models.

Collectively, these techniques synergise: LDA preprocesses for SVM/NN inputs, fostering ensemble models that deliver superior Sharpe ratios. By tuning hyperparameters like SVM’s C or NN hidden sizes via cross-validation, practitioners realise consistent outperformance, turning market inefficiencies into quantifiable edges. This ML arsenal not only boosts yields but also scales to big data, positioning finance professionals at the forefront of quantitative innovation.

6 Learn More

This section compiles journal references from the techniques covered, highlighting empirical applications in finance, alongside curated resources emphasising the strengths of machine learning algorithms—such as enhanced predictive power, non-linear modelling, and scalability to big data—over traditional methods.

Journal references include: LASSO Regression [?], which thresholds covariances for high-dimensional estimation in econometric models; k-Means Clustering [?], diversifying portfolios via out-of-sample optimisation; LDA [?], classifying financial distress with discriminant projections; SVMs [?], predicting volatile stock prices through regression kernels; and NNs [?], optimising portfolios via sensitivity-based risk factors.

For further reading on ML strengths:

- Gao, Hanyao, et al. “Machine Learning in Business and Finance: A Literature Review and Research Opportunities.” *Financial Innovation*, vol. 10, no. 1, 2024, Springer Nature, doi:10.1186/s40854-024-00629-z. (Emphasises deep learning’s superiority in handling unstructured data for forecasting and decision-making.)
- Sarker, Iqbal H. “Machine Learning: Algorithms, Real-World Applications and Research Directions.” *SN Computer Science*, vol. 2, no. 3, 2021, Springer Nature, doi:10.1007/s42979-021-00592-x. (Highlights ensemble methods’ reliability in fraud detection and risk management.)
- “Machine Learning in Finance: Benefits and Use Cases.” *KeenEthics*, 9 June 2023, keenethics.com/blog/machine-learning-in-finance. (Stresses automation’s cost savings and fraud prevention via anomaly detection.)

7 Comparing Models

The following table compares the models across key features relevant to financial applications, summarising strengths and limitations based on their designs and empirical performance.

8 Comparing Models

The following table compares the models across key features relevant to financial applications, summarising strengths and limitations based on their designs and empirical performance.

Feature	LASSO	k-Means	Trees	LDA	SVM	NN
Supervised/Unsuper.	Super.	Unsuper.	Super.	Super.	Super.	Super.
Missing Data	Poorly (impute)	Poorly (impute)	Well (surr. splits)	Poorly (impute)	Poorly (impute)	Poorly (impute)
High-Dim Scalable	Well (L1 sparse)	Well (iter.)	Well (part.)	Well (reduce)	Mod. (quad.)	Well (reg.)
Interpretable	Well (coefs)	Mod. (cents)	Well (paths)	Well (proj.)	Poorly (kern.)	Poorly (box)
Non-Linear	Poorly (lin.)	Mod. (spher.)	Well (splits)	Poorly (lin.)	Well (kern.)	Well (act.)
Outlier Robust	Mod. (shrink)	Poorly (skew)	Well (isol.)	Poorly (norm.)	Well (marg.)	Mod. (drop)
Comp. Efficiency	Well (convex)	Well (O(n))	Well (O(n log n))	Well (eigen)	Poorly (quad.)	Poorly (iter.)
Feat. Scaling	Yes	No	No	Yes	Yes	Yes
Multiclass	Yes (ext.)	N/A	Yes (splits)	Yes	Yes (1v1)	Yes
Prob. Outputs	No	No	Yes (ext.)	Yes	No (ext.)	Yes (soft.)

Table 1: Comparison of ML Models for Trading Strategies